

Benchmark de Algoritmos de Ordenação

Édipo Elder Fernandes de Melo

04/10/2026

1 Introdução

Este relatório apresenta uma análise comparativa de cinco algoritmos de ordenação aplicados a uma estrutura de dados em C++ (`std::vector<int>`). O objetivo é medir, de forma padronizada, o tempo de execução médio (em milissegundos) de cada algoritmo em função do tamanho do vetor, considerando três cenários distintos: melhor caso, pior caso e caso médio.

O melhor caso é quando o vetor já está ordenado. O pior caso é quando o vetor está ordenado de forma decrescente. Para testar o caso médio, optamos em criar um vetor com valores aleatórios. Neste último caso de cenário, é possível que o vetor tenha valores repetidos.

1.1 Os algoritmos avaliados

A seguir, resumiremos as características de cada algoritmo testado:

Insertion Sort. Percorre o vetor da esquerda para a direita, inserindo cada elemento na posição correta dentro da parte já ordenada. É eficiente para vetores pequenos ou quase ordenados, mas quadrático no pior caso.

Selection Sort. A cada iteração, seleciona o menor elemento da parte não ordenada e o coloca na posição correta. Realiza sempre o mesmo número de comparações, independentemente da ordem inicial do vetor.

Bubble Sort. Compara pares adjacentes e troca-os se estiverem fora de ordem, repetindo o processo até que nenhuma troca seja necessária. Possui uma otimização que interrompe a execução precocemente se o vetor já estiver ordenado.

Quick Sort. Escolhe um pivô (sempre no meio do range, neste experimento), particiona o vetor em torno dele e ordena recursivamente as subpartes. É um dos algoritmos mais eficientes na prática, com complexidade média $O(n \log n)$.

Merge Sort. Divide o vetor recursivamente pela metade, ordena cada metade e depois as intercala. Garante complexidade $O(n \log n)$ em todos os casos, mas utiliza memória auxiliar.

A tabela a seguir resume a complexidade dos algoritmos no melhor, pior e caso médio:

Tabela 1: Complexidade dos algoritmos de ordenação avaliados

Algoritmo	Melhor	Médio	Pior	Estável?
Insertion Sort	$O(n)$	$O(n^2)$	$O(n^2)$	Sim
Selection Sort	$O(n^2)$	$O(n^2)$	$O(n^2)$	Não
Bubble Sort	$O(n)$	$O(n^2)$	$O(n^2)$	Sim
Quick Sort	$O(n \log n)$	$O(n \log n)$	$O(n^2)$	Não
Merge Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	Sim

1.2 Cenário do experimento

O programa foi executado num processador **MacBook M4** rodando **MacOS 27**, compilado com Apple clang versão 21.0.0 (clang-2100.3.34.2) e os seguintes parâmetros:

```
g++ -std=c++23 -march=native -mtune=native -fno-stack-protector benchmark_sort.cpp -o bench_sort
```

O parâmetro `-march=native -mtune=native` diz ao compilador para usar instruções específicas da sua CPU e `-fno-stack-protector` remove canários de pilha (reduz overhead por chamada).

Em todos os testes:

- Foram criados três vetores com tamanho `VECTOR_SIZE` (10000), utilizados de forma padrão nos testes, sendo preenchidos de formas distintas:
 - **Melhor caso:** vetor já ordenado (`melhor_caso[i] = i`).
 - **Pior caso:** vetor em ordem inversa (`pior_caso[VECTOR_SIZE - i - 1] = i`).
 - **Caso médio:** vetor aleatório (`rand() % VECTOR_SIZE`).
- Os tamanhos de vetor testados variam de 500 a 10000, em passos de 500.
- Cada medição foi repetida 10 vezes (`TRIALS`), e calculamos a média dos tempos obtidos.
- Para evitar que o algoritmo receba um vetor já ordenado nos cenários de testes do pior caso e caso médio, a cada nova execução do algoritmo foi criada uma cópia do vetor padrão.

2 A função de benchmark

A fim de evitar repetição de código para cada algoritmo implementado, padronizamos a execução dos testes com um função genérica:

```
typedef void (*algoritmo_ordenacao)(vector<int>& vetor, int end);

inline long benchmark(algoritmo_ordenacao algoritmo, vector<int> &vetor, int end) {
    ...
}
```

A implementação dos algoritmos foi feita:

1. **Com a mesma assinatura de parâmetros.** Todas as funções recebem (`vetor, end`) e retornam `void`. O ponteiro de função `algoritmo_ordenacao` permite chamá-las polimorficamente.
2. **Medição isolada.** O cronômetro só envolve a chamada do algoritmo em si. As instruções para criação da cópia do vetor padrão não são computadas no teste.
3. **Cópia do vetor a cada teste.** Isso evita que o algoritmo receba um vetor já ordenado na execução seguinte.

O `main` cria os três vetores de teste e, para cada função registrada na tabela de `funcoes`, computa a média de tempo de 3 repetições para diferentes tamanhos, gerando um arquivo CSV diretamente utilizável pelo R. Optamos por fazer uma média de 3 execuções uma vez que os algoritmos Bubble Sort e Selection sort levam de 25 a quase 200 segundos de execução.

3 Análise por algoritmo

3.1 Insertion Sort

No melhor caso (vetor já ordenado), o algoritmo percorre o vetor uma única vez, resultando em crescimento linear. No pior caso (ordem inversa) e no caso médio, o comportamento é quadrático, com o pior caso apresentando tempos significativamente maiores.

```
void insertion_sort(vector<int>& vetor, int end){
    int key, holePos;
    // percorrer parte não-ordenada (começa em 1)
    for (int i = 1; i < end; i++) {
        key = vetor[i];
        holePos = i - 1;
```

```

// percorrer (reverso) a parte ordenada até o início ou achar a posição
while (holePos >= 0 && vetor[holePos] > key) {
    vetor[holePos + 1] = vetor[holePos];
    holePos = holePos - 1;
}
// inserir item na posição criada
vetor[holePos + 1] = key;
}
}

```

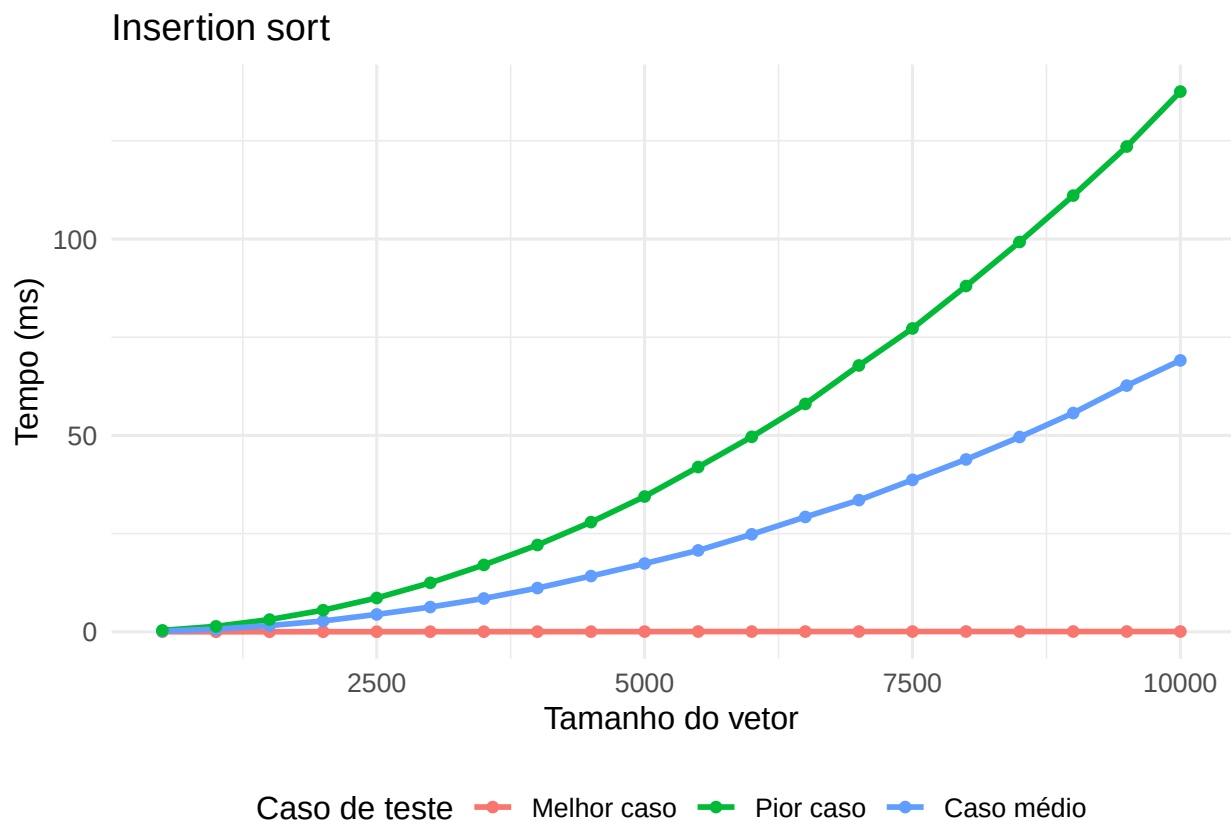


Figura 1: Insertion Sort

3.2 Selection Sort

O Selection Sort realiza sempre o mesmo número de comparações, independentemente da ordem inicial. Contudo, a instrução que determina a troca de valores no vetor (`if (vetor[i] > vetor[j])`) gera um impacto maior quando o vetor contém dados aleatórios, conforme pode ser observado na Figura 2.

```

void selection_sort(vector<int>& vetor, int end){
    for (int i = 0; i < end - 1; i++)
        for (int j = i + 1; j < end; j++)
            if (vetor[i] > vetor[j])
                swap(vetor[i], vetor[j]);
}

```

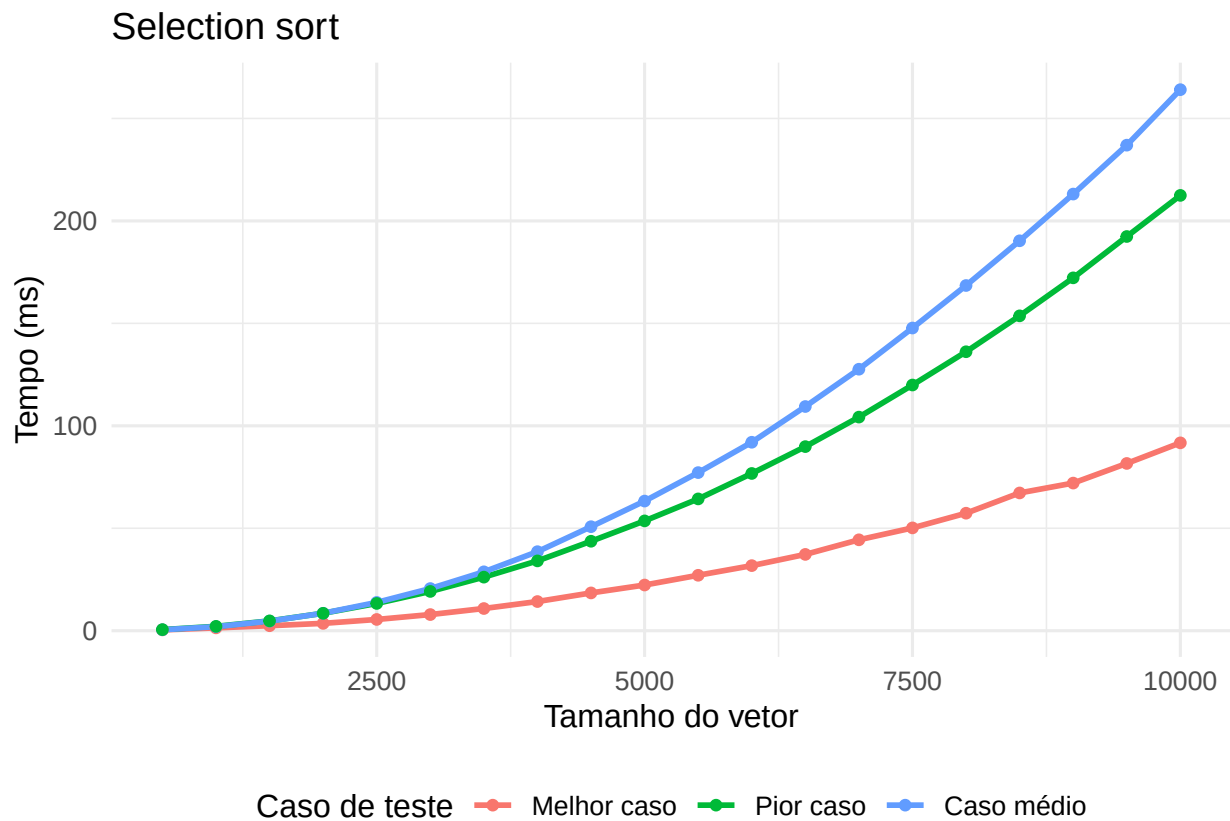


Figura 2: Selection Sort

3.3 Bubble Sort

O Bubble Sort com a otimização de parada precoce apresenta comportamento linear no melhor caso (vetor já ordenado). Nos demais casos, o crescimento é quadrático. AS “quebras” no gráfico do melhor caso podem ser consequência da execução concorrente com outros programas do sistema operacional.

```
void bubble_sort(vector<int>& vetor, int end){
    int j = end - 1;
    bool houveTroca = false;
    do {
        houveTroca = false;
        for (int i = 0; i <= j; i++) {
            if (vetor[i + 1] < vetor[i]) {
                swap(vetor[i], vetor[i + 1]);
                houveTroca = true;
            }
        }
        j = j - 1;
    } while (houveTroca);
}
```

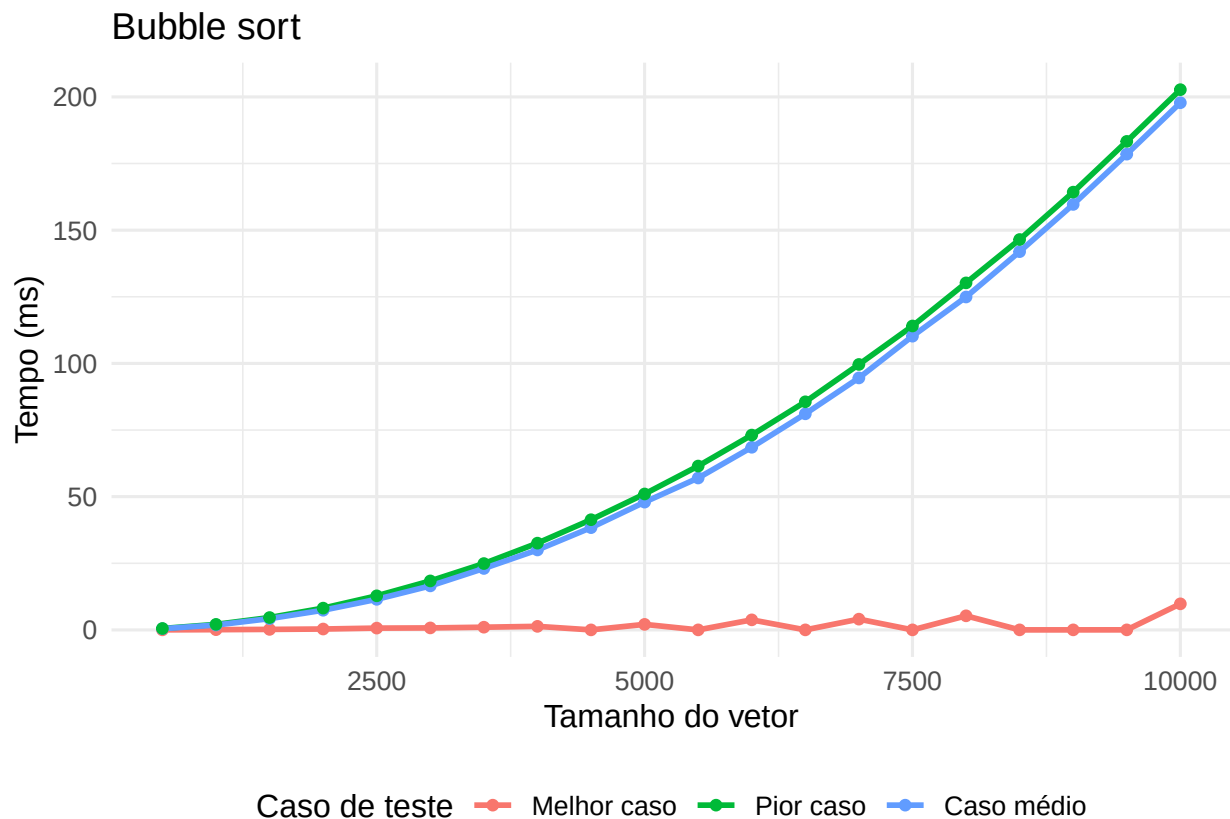


Figura 3: Bubble Sort

3.4 Quick Sort

A escolha do pivô pode interferir no desempenho do algoritmo. Geralmente, é recomendado a escolha de um pivô aleatório. Neste estudo, optamos em escolher o pivô sempre no meio do range para manter uma constância maior entre a execução com vetores de tamanhos diferentes.

O Quick Sort apresenta comportamento $O(n \log n)$ em todos os casos. Observa-se que o pior caso teórico (vetor já ordenado) não se manifesta de forma significativa e a curva fica próxima do melhor caso. Já para o caso aleatório, pela opção de escolha do pivô sempre no meio, a curva foi acima do pior caso. Um fator que pode estar causando esta diferença é a predição do pipeline da CPU (neste caso, um Apple M4).

```
int particao(vector<int>& vetor, int start, int end){
    // int pivo = start + rand() % (end - start + 1); // aleatório entre 0 e end
    //int pivo = end; // pivô sempre no final
    int pivo = start + (end - start) / 2; // pivô sempre no meio
    int x = vetor[pivo];
    swap(vetor[pivo], vetor[end]);
    int i = start - 1;
    for (int j = start; j < end; j++) {
        if (vetor[j] <= x) {
            i++;
            swap(vetor[i], vetor[j]);
        }
    }
    swap(vetor[i + 1], vetor[end]);
    return i + 1;
}

void quick_sort(vector<int>& vetor, int start, int end){
```

```

    if (start < end) {
        int idx = particao(vetor, start, end);
        quick_sort(vetor, start, idx - 1);
        quick_sort(vetor, idx + 1, end);
    }
}

void quick_sort(vector<int>& vetor, int end){
    quick_sort(vetor, 0, end);
}

```

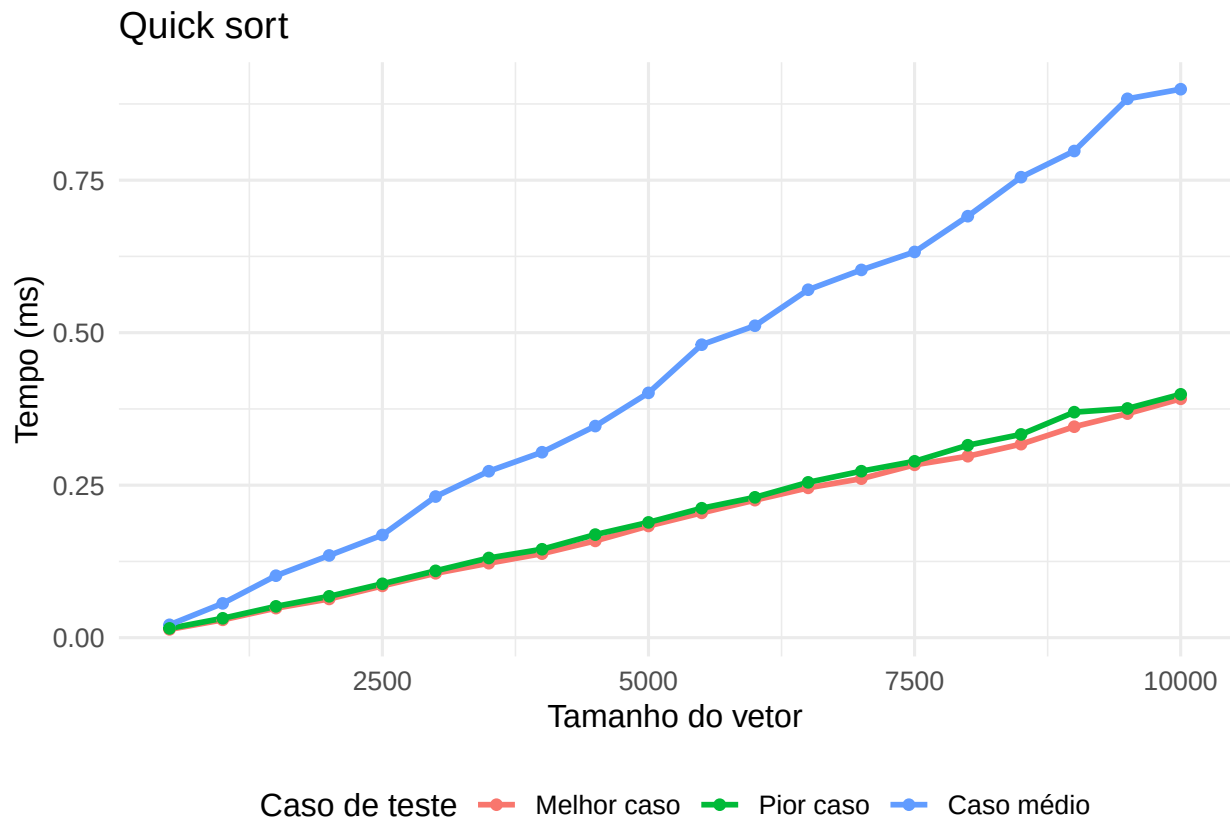


Figura 4: Quick Sort

3.5 Merge Sort

O Merge Sort também apresenta comportamento $O(n \log n)$ em todos os casos, com curvas praticamente coincidentes. Seu tempo é ligeiramente superior ao do Quick Sort devido ao uso de memória auxiliar. De mesma forma, para o caso médio, a curva de tempo de execução é ligeiramente superior.

```

// método para fazer o join dos vetores ordenados
void intercala(vector<int>& left, vector<int>& right, vector<int>& vetor) {
    int i = 0; // índice para left
    int j = 0; // índice para right
    int k = 0; // índice para o vetor resultado
    // Compara elementos e insere o menor em vetor
    while (i < left.size() && j < right.size()) {
        if (left[i] <= right[j]) {
            vetor[k++] = left[i++];
        } else {
            vetor[k++] = right[j++];
        }
    }
}

```

```

    }
    // Copia os elementos restantes
    while (i < left.size()) {
        vetor[k++] = left[i++];
    }
    while (j < right.size()) {
        vetor[k++] = right[j++];
    }
}

// rotina recursiva do merge
void merge_sort(vector<int>& vetor, int end) {
    if (end < 2) return;
    int meio = end / 2;
    // vetores esquerda (left) e direita (right)
    vector<int> left(meio);
    vector<int> right(end - meio);
    // Copia os elementos para left e right
    for (int i = 0; i < meio; i++) {
        left[i] = vetor[i];
    }
    for (int i = meio; i < end; i++) {
        right[i - meio] = vetor[i];
    }
    // Chamadas recursivas
    merge_sort(left, meio);
    merge_sort(right, end - meio);
    // Intercala de volta ao vetor
    intercala(left, right, vetor);
}

```

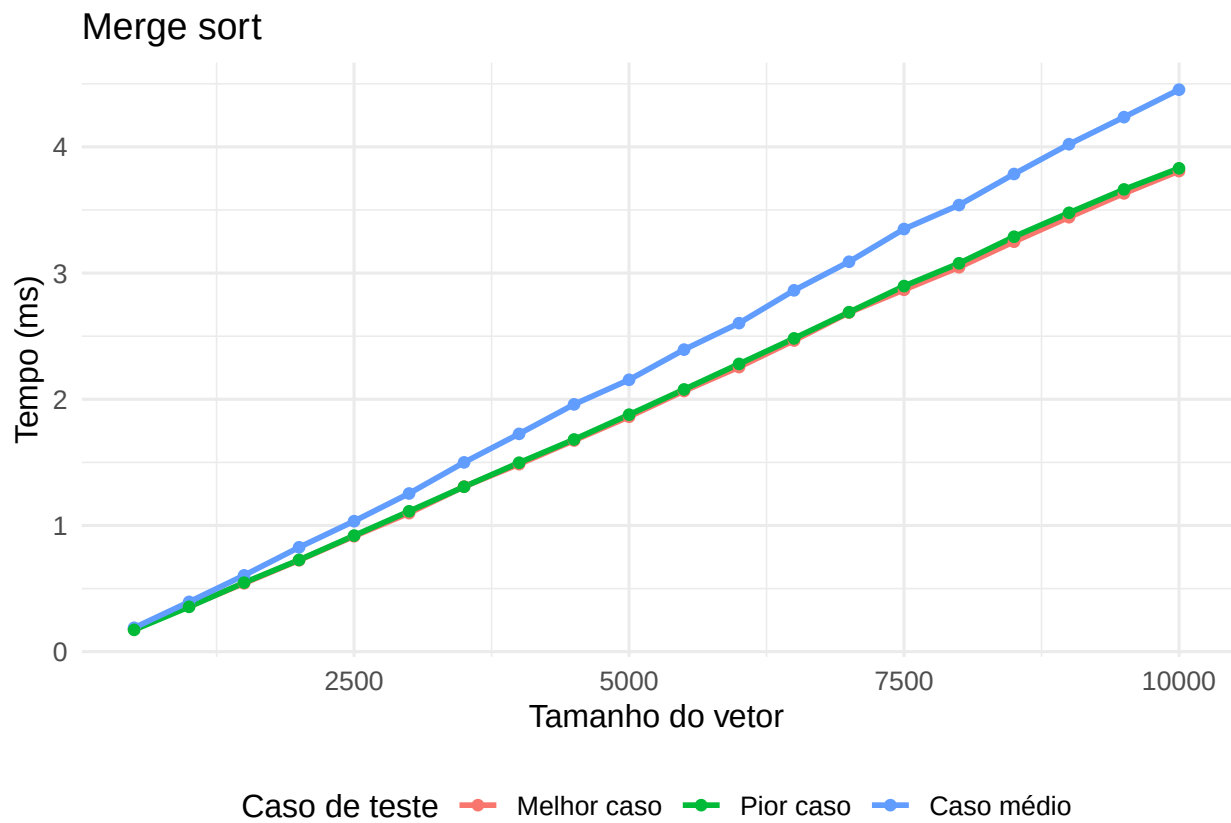


Figura 5: Merge Sort

4 Conclusão

4.1 Melhor caso (vetor já ordenado)

No melhor caso, Insertion Sort e Bubble Sort apresentam crescimento linear, enquanto Selection Sort, Quick Sort e Merge Sort mantêm seus comportamentos característicos. O Insertion Sort é o mais rápido nesse cenário.

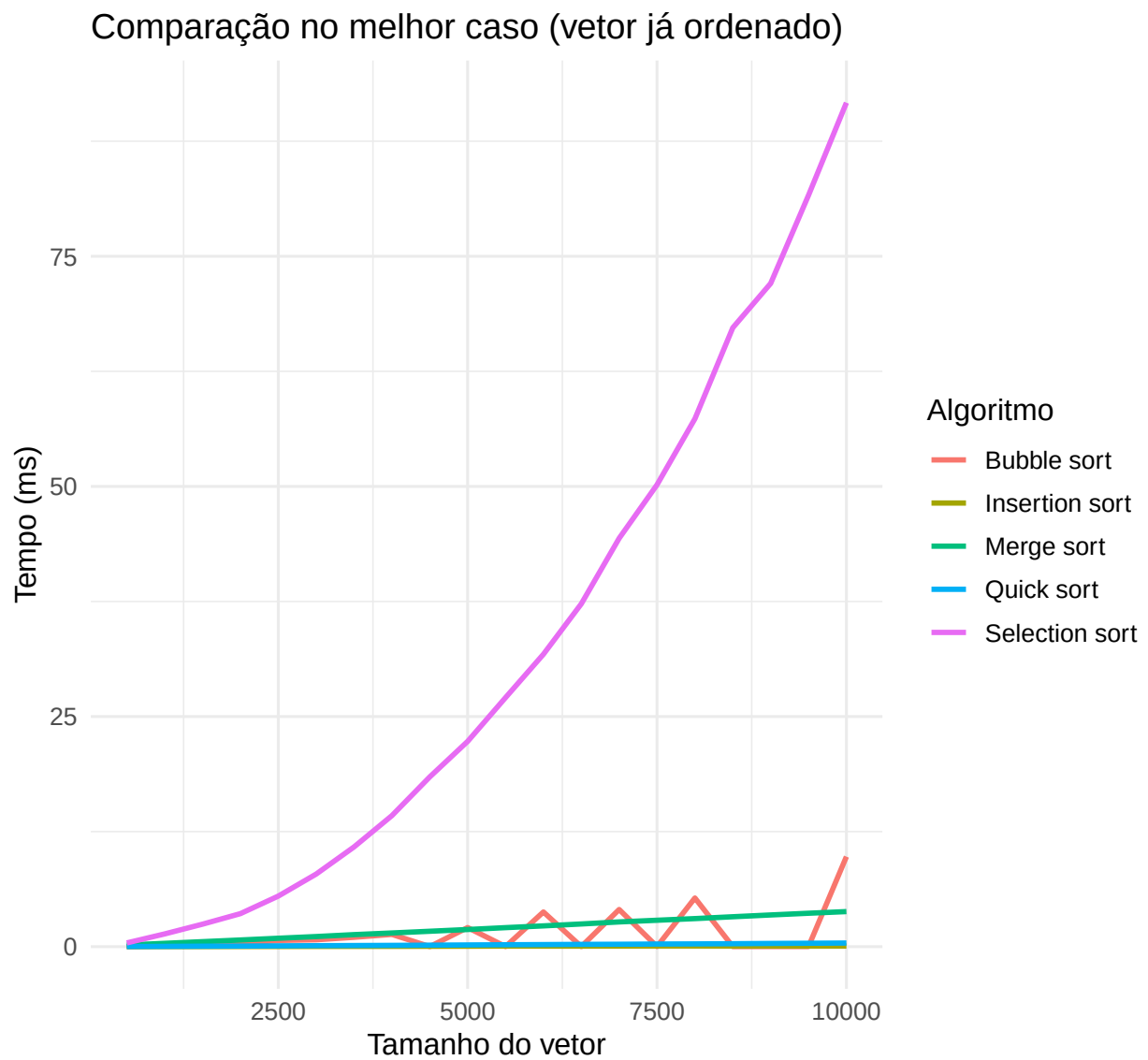


Figura 6: Comparação no melhor caso

4.2 Pior caso (vetor em ordem inversa)

No pior caso, Insertion Sort, Selection Sort e Bubble Sort apresentam crescimento quadrático acentuado, com tempos muito superiores aos do Quick Sort e Merge Sort, que se mantêm em $O(n \log n)$.

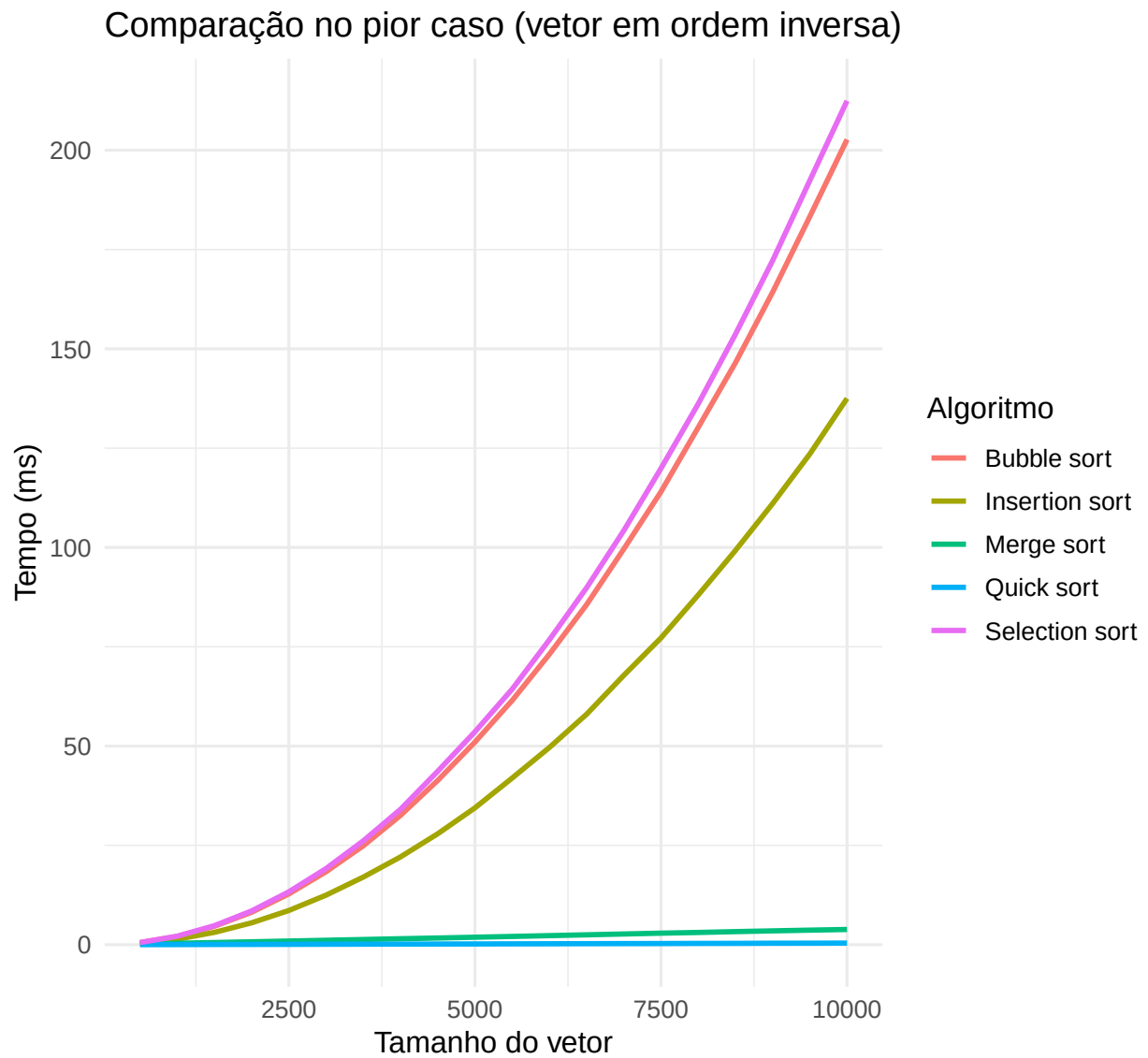


Figura 7: Comparação no pior caso

4.3 Caso médio (vetor aleatório)

No caso médio, o comportamento é semelhante ao pior caso para os algoritmos quadráticos, mas com tempos ligeiramente menores. Quick Sort e Merge Sort continuam sendo as melhores escolhas para vetores grandes.

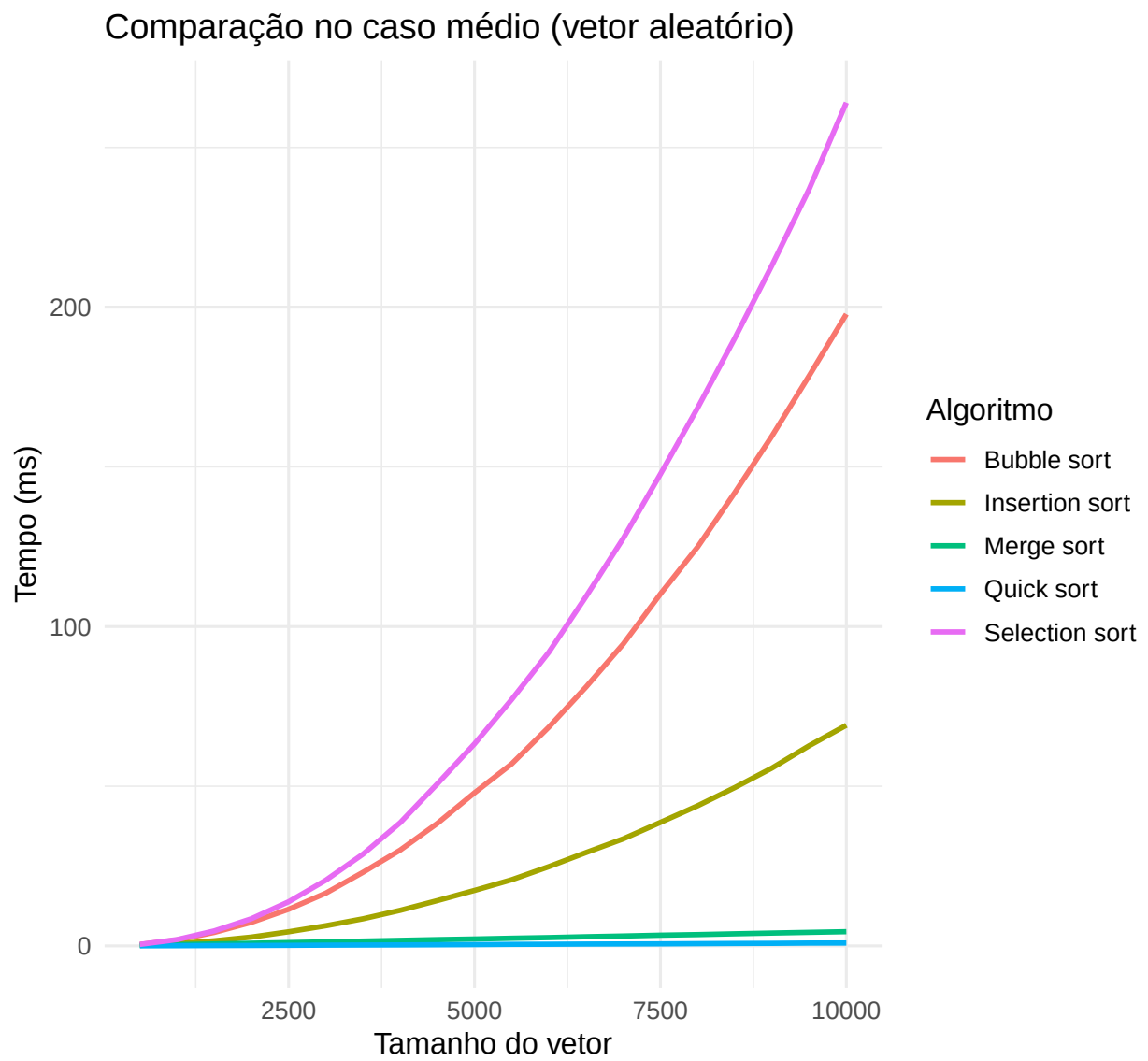


Figura 8: Comparação no caso médio

Algoritmos $O(n \log n)$: Quick Sort e Merge Sort

Melhor, pior e caso médio

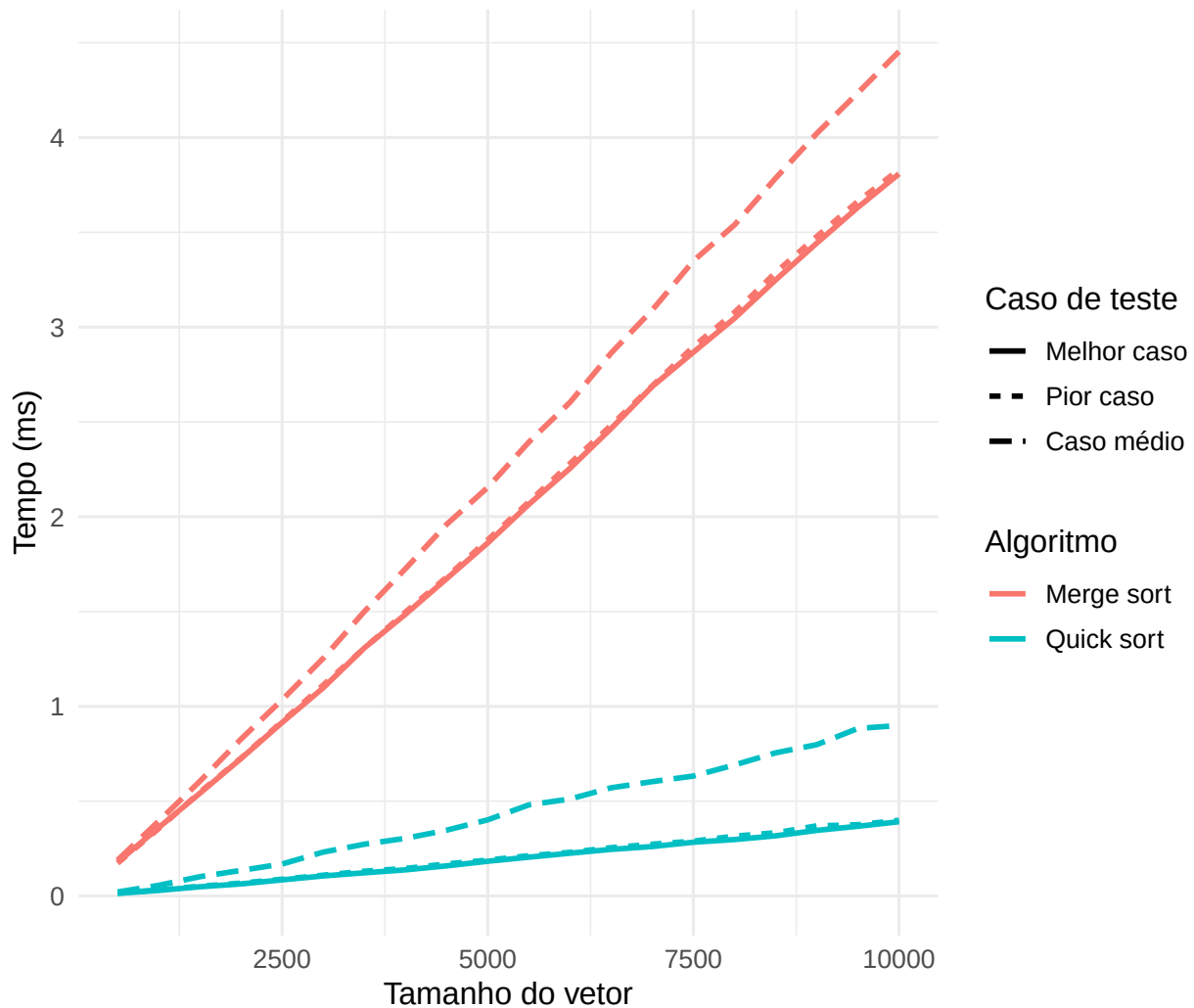


Figura 9: Comparação entre os algoritmos $O(n \log n)$

4.4 Considerações finais

Observamos que, empiricamente, o algoritmo **Quick sort** tem comportamento distinto do comportamento esperado na teoria, com o caso médio ficando acima do pior caso (Figura 4). Supomos que deve-se aos parâmetros utilizados com o compilador (otimização, instruções específicas da CPU, etc.), bem como a predição de instruções do pipeline e cache de instruções pela CPU. CAberia

De modo geral, observamos que, para vetores grandes e desordenados, o **Quick Sort** e o **Merge Sort** são as melhores escolhas, com complexidade $O(n \log n)$ em todos os casos. Na prática, o **Quick sort** é mais rápido que o **Merge sort** (Figura 9), além da vantagem de utilizar menos memória.

O **Insertion Sort** se destaca no melhor caso (vetor já ordenado) com comportamento linear, sendo útil quando os dados estão quase ordenados. Já o **Selection Sort** e o **Bubble Sort**, embora sejam de simples implementação, são ineficientes para vetores grandes, apresentando crescimento quadrático mesmo no melhor caso (no caso do Selection Sort).

Como trabalho futuro, sugerimos um estudo do algoritmo **Quick Sort** com variação do método de escolha do pivô (no fim, no meio, aleatório, etc) bem como quais parâmetros de compilação são os mais indicados para programas que utilizem este algoritmo.

A Tabela de tempos

Tabela 2: Tempos médios de execução (em milissegundos) por algoritmo, caso e tamanho

Tam.	Caso	Insert	Select	Bubble	Quick	Merge
500	Melhor caso	0.004508	0.406866	0.022562	0.013550	0.178058
1000	Melhor caso	0.008995	1.381279	0.085746	0.029237	0.360470
1500	Melhor caso	0.013149	2.441887	0.187974	0.048433	0.541525
2000	Melhor caso	0.017279	3.586541	0.332674	0.063483	0.724550
2500	Melhor caso	0.022495	5.492483	0.673633	0.084895	0.914633
3000	Melhor caso	0.026354	7.885821	0.744570	0.105541	1.097645
3500	Melhor caso	0.033237	10.834641	1.016458	0.122091	1.307854
4000	Melhor caso	0.035729	14.235525	1.331379	0.137766	1.485558
4500	Melhor caso	0.042550	18.442920	0.007679	0.158679	1.673204
5000	Melhor caso	0.050345	22.303054	2.073579	0.183125	1.861645
5500	Melhor caso	0.063749	27.056524	0.009291	0.204645	2.065287
6000	Melhor caso	0.060566	31.765604	3.767604	0.225462	2.255220
6500	Melhor caso	0.060470	37.244112	0.011283	0.245604	2.465408
7000	Melhor caso	0.062862	44.366616	4.035995	0.260645	2.686733
7500	Melhor caso	0.072691	50.177795	0.012479	0.283354	2.867458
8000	Melhor caso	0.072187	57.349112	5.285804	0.297474	3.046441
8500	Melhor caso	0.066629	67.234766	0.013941	0.317237	3.248479
9000	Melhor caso	0.070316	72.052654	0.014695	0.345866	3.442616
9500	Melhor caso	0.066933	81.635383	0.015595	0.367329	3.631379
10000	Melhor caso	0.067333	91.685895	9.779508	0.391579	3.808116
500	Pior caso	0.354866	0.531970	0.529350	0.015225	0.172737
1000	Pior caso	1.386537	2.132287	2.076808	0.031783	0.355237
1500	Pior caso	3.106895	4.788462	4.640129	0.051345	0.547029
2000	Pior caso	5.504754	8.505604	8.169641	0.067808	0.727104
2500	Pior caso	8.609729	13.313574	12.786854	0.088329	0.919954
3000	Pior caso	12.473629	19.170529	18.380366	0.109587	1.112520
3500	Pior caso	17.022387	26.149429	24.899946	0.130641	1.306845
4000	Pior caso	22.098962	34.074420	32.553241	0.144970	1.497408
4500	Pior caso	27.923949	43.661195	41.350858	0.169000	1.681570
5000	Pior caso	34.436708	53.596370	51.003754	0.189008	1.877941
5500	Pior caso	41.946020	64.324824	61.484116	0.212216	2.078129
6000	Pior caso	49.639179	76.742983	73.109270	0.230087	2.280187
6500	Pior caso	58.004254	89.846741	85.594787	0.254662	2.482433
7000	Pior caso	67.798924	104.246971	99.609145	0.272933	2.690045
7500	Pior caso	77.229641	119.914216	114.042224	0.289145	2.897079
8000	Pior caso	88.026233	136.144958	130.212516	0.315491	3.077445
8500	Pior caso	99.240687	153.665662	146.484650	0.333229	3.287979
9000	Pior caso	111.048946	172.213279	164.269816	0.369687	3.477633
9500	Pior caso	123.537008	192.389016	183.307933	0.375766	3.662937
10000	Pior caso	137.525325	212.448312	202.710633	0.398954	3.830404
500	Caso médio	0.171541	0.468912	0.453916	0.021120	0.189245
1000	Caso médio	0.677404	1.977487	1.833133	0.056233	0.394541
1500	Caso médio	1.591454	4.691333	4.227187	0.101612	0.604675
2000	Caso médio	2.755091	8.516541	7.380087	0.134808	0.827158
2500	Caso médio	4.418495	13.823025	11.470375	0.168316	1.034129
3000	Caso médio	6.304820	20.575516	16.512691	0.231362	1.253462
3500	Caso médio	8.485929	28.744725	23.049849	0.272895	1.500016
4000	Caso médio	11.136670	38.578641	30.001474	0.304020	1.725808
4500	Caso médio	14.190596	50.763721	38.371820	0.346908	1.959666
5000	Caso médio	17.373804	63.275841	47.943525	0.401337	2.154429
5500	Caso médio	20.715058	77.165937	57.000570	0.480325	2.393483

6000	Caso médio	24.820899	91.977221	68.528196	0.511375	2.603191
6500	Caso médio	29.235912	109.420258	81.083067	0.570408	2.863192
7000	Caso médio	33.518562	127.622345	94.547637	0.602891	3.089829
7500	Caso médio	38.676662	147.728900	110.214700	0.632391	3.349104
8000	Caso médio	43.875908	168.475279	124.918058	0.690966	3.538941
8500	Caso médio	49.580404	190.292575	141.935446	0.754937	3.785116
9000	Caso médio	55.688241	213.101745	159.653049	0.797817	4.021012
9500	Caso médio	62.675754	236.965570	178.517708	0.883404	4.235641
10000	Caso médio	69.083770	264.043154	197.811458	0.899141	4.453229
